

编译原理-语法分析 06

LR 分析法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 4 月 10 日



福州大学
FUZHOU UNIVERSITY 1958



- ① 课前复习
- ② 移入/归约冲突
- ③ LR 分析法
- ④ LR(0)



① 课前复习

自顶向下的语法分析器
自底向上的关键概念

② 移入/归约冲突

③ LR 分析法

④ LR(0)



① 课前复习

自顶向下的语法分析器
自底向上的关键概念

② 移入/归约冲突

③ LR 分析法

④ LR(0)

语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

自顶向下 (Top-Down) 语法分析

特点:

- 从**开始符号**出发, 寻找“匹配”的**推导**。
- 推导: 根据文法的产生式规则, 把串中出现的产生式的左部符号替换成右部。
- 从**根**开始, 构造语法树。
- 具体实现: 递归下降分析法、预测分析法。

表驱动的预测分析法步骤:

- ① 处理文法: 左递归、回溯、二义性;
- ② 计算 FIRST 和 FOLLOW;
- ③ 计算预测分析表 M ;
- ④ 计算同步集合用于错误处理等;
- ⑤ 处理输入。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号 \$）；
 - 每个单元格里填写对应的产生式，或者错误标记；
 - 单元格 (i, j) 的内容表示若在匹配第 i 行的非终结符时遇到第 j 列的终结符时，该确定性地选用的生成式。

预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;

如何处理错误?

- ① 恐慌模式 (Panic mode): 忽略输入流中的一系列符号, 直到出现符合设计者预期的词元;
- ② 短语恢复: 改变、插入或删除输入中的符号, 并给出适当的错误信息。

预测分析法

错误恢复

如何检测错误?

- ① 栈顶的终结符号与输入不匹配;
- ② 栈顶的非终结符号 A , 但 $M[A, a] = error$ 。

如何处理错误?

- ① 恐慌模式 (Panic mode): 忽略输入流中的一系列符号, 直到出现符合设计者预期的词元;
- ② 短语恢复: 改变、插入或删除输入中的符号, 并给出适当的错误信息。

恐慌模式的错误处理

定义

同步集合 (synchronization set): 错误处理函数/过程会不断丢弃输入的符号直到遇到一个同步集合内的符号。

如何构造同步集合?

- ① 将 FOLLOW(A) 里的元素都加入同步集合;
- ② 将更高层次的非终结符号的 FOLLOW 集合也加入同步集合中;
- ③ 将 FIRST(A) 的符号也加入同步集合;
- ④ 如果非终结符可以生成 ϵ , 则使用它作为默认值;
- ⑤ 直接把不匹配的终结符号弹出;



① 课前复习

自顶向下的语法分析器
自底向上的关键概念

② 移入/归约冲突

③ LR 分析法

④ LR(0)

自底向上 (Bottom-Up) 语法分析

- 从分析树的底部(叶节点) 向顶部(根节点) 方向构造分析树, 可以看成是将输入串 w **归约**为文法开始符号 S 的过程。
- 自顶向下的语法分析采用**最左推导**方式。
- 自底向上的语法分析采用**最左归约**方式 (反向构造**最右推导**)。

定义 (归约, Reduce)

归约: 先将一个子串与某规则的右部匹配, 然后将该子串替换为该规则的左部。

自底向上语法分析的核心问题

- ① 什么时候进行归约？归约到什么程度？

例

$a+b*c$

- ② 选择哪个产生式进行归约？

例

```
int x=5;
```

自底向上语法分析通用框架

定义 (移入-归约分析 (Shift-Reduce Parsing))

- 在对输入串的一次从左到右扫描过程中，语法分析器将零个或多个输入符号移入到栈的顶端，直到它可以对栈顶的一个文法符号串 β 进行归约为止；
- 然后，它将 β 归约为某个产生式的左部；
- 语法分析器不断地重复这个循环，直到它检测到一个语法错误，或者栈中包含了开始符号且输入缓冲区为空¹为止。

¹当进入这样的格局时，语法分析器停止运行，并宣称成功完成了语法分析。

移入-归约分析示例

例

给定文法：

$$S \rightarrow aAcBe$$

$$A \rightarrow b$$

$$A \rightarrow Ab$$

$$B \rightarrow d$$

试分析 *abbcde*

最左规约——最右推导的逆过程：

$$S \Rightarrow aAcBe \Rightarrow aAcde \Rightarrow aAbcde \Rightarrow abbcde$$

移入-归约分析的四种动作

移入-归约分析器可采取的 4 种动作：

- ① 移入：将下一个输入符号移到栈的顶端。
- ② 归约：被归约的符号串的**右端**必然处于栈顶。语法分析器在栈中确定这个串的**左端**，并决定用哪个非终结符来替换这个串。
- ③ 接受：宣布语法分析过程成功完成。
- ④ 报错：发现一个语法错误，并调用错误恢复子例程。

移入-归约分析

核心问题

例

给定文法：

$$S \rightarrow aAcBe$$

$$A \rightarrow b$$

$$A \rightarrow Ab$$

$$B \rightarrow d$$

当栈内元素为 aAb 时调用哪一条规则进行规约？

基本概念

对于文法 $G(S)$, $\alpha\beta\delta$ 是 G 的一个句型。如果有 $S \Rightarrow_{rm}^* \alpha A \delta$, 且

- $A \Rightarrow_{rm}^+ \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的短语。
- $A \Rightarrow_{rm} \beta$, 则称 β 是句型 $\alpha\beta\delta$ 相对于 A 的直接短语（简单短语）。
- 一个句型的最左直接短语称为该句型的一个句柄（Handle）。
- 注：如果文法是二义的, 则最右推导不止一个, 可能存在多个句柄。如果文法没有二义, 那么只有唯一的句柄。
- 找句柄的两个方法：
 - ① 写出从开始符号到句型的最右推导, 然后找出最后一步与之前一步的区别;
 - ② 构建语法树, 找出当前句型的最左高度为 1 的子树。

经典例子

例

用下列文法分析 $i_1 + i_2 * i_3$:

$$E \rightarrow T \mid E + T \quad T \rightarrow F \mid T * F \quad F \rightarrow (E) \mid i$$

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * i_3 \\ &\Rightarrow E + F * i_3 \Rightarrow E + i_2 * i_3 \Rightarrow T + i_2 * i_3 \\ &\Rightarrow F + i_2 * i_3 \Rightarrow i_1 + i_2 * i_3 \end{aligned}$$

对于句型 $i_1 + i_2 * i_3$ 而言,

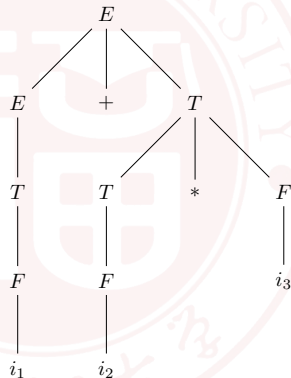
- 短语有 $i_1, i_2, i_3, i_2 * i_3, i_1 + i_2 * i_3$;
- 直接短语有 i_1, i_2, i_3 ;
- 句柄是 i_1 。

如何通过语法树找短语、句柄

例

在一个句型的语法树中

- 任一子树的叶节点从左到右排列所组成的符号串都是该句型的短语。
- 任一最小子树（树高为 1）的叶节点所组成的符号串都是该句型的直接短语。
- 句柄是最左边的高度为 1 的子树的叶子结点。



规范句型

- **规范归约** $\equiv$ **最左规约**：在自底向上的分析中，总是采用最左归约的方式，因此把最左归约称为规范归约。
- **规范推导** $\equiv$ **最右推导**：规范归约是最右推导的逆过程，则最右推导相应地称为规范推导。
- **规范句型**：由规范推导推出的句型称为规范句型。

前缀、活前缀、可归前缀

- ① 前缀：符号串的任意首部。
- ② 活前缀：文法 $G(S)$ 的一个规范句型 $\alpha\beta$ ，其中 β 为终结符号串。若 α 不含句柄之后的任何符号，则称 α 为**活前缀（可行前缀）**。
- ③ 如果 α 是含有句柄的活前缀，则称 α 为**可归前缀**。
- ④ **可归前缀即最长的活前缀。**



① 课前复习

② 移入/归约冲突

③ LR 分析法

④ LR(0)

有些文法不能使用移入-归约分析框架进行语法分析。它们会产生如下冲突：

- 移入/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，也无法判断应该将新元素移入栈还是将栈中已有元素进行归约。

例

if 语句的抽象文法

$$S \rightarrow iEtSeS \mid iEtS \mid o$$

当栈内元素为 $iEt iEt S$ ，且下个元素为 e 时，应当将栈内元素先进行归约，还是移入 e 等待下一个 S 出现后再归约呢？

归约/归约冲突

有些文法不能使用移入-归约分析框架进行语法分析。它们会产生如下冲突：

- 移入/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，也无法判断应该将新元素移入栈还是将栈中已有元素进行归约。
- 归约/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，用哪个产生式归约。

例

假设有文法

$$\begin{aligned} S &\rightarrow P; \\ S &\rightarrow E := E; \\ P &\rightarrow i(L) \\ L &\rightarrow L, i \mid i \\ E &\rightarrow i(D) \\ D &\rightarrow D, i \mid i \end{aligned}$$

当前栈中元素依次为 i (i 。应当如何规约？

- 如果 i 对应的符号表示一个过程，则应该调用 $L \rightarrow i$ 进行归约；
- 如果 i 对应为一个数组，则应该调用 $D \rightarrow i$ 来归约；

归约/归约冲突

有些文法不能使用移入-归约分析框架进行语法分析。它们会产生如下冲突：

- 移入/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，也无法判断应该将新元素移入栈还是将栈中已有元素进行归约。
- 归约/归约冲突：即使知道了所有栈内符号和后续的 k 个符号，用哪个产生式归约。

例

假设有文法

$S \rightarrow P;$
 $S \rightarrow E := E;$
 $P \rightarrow i(L)$
 $L \rightarrow L, i \mid i$
 $E \rightarrow i(D)$
 $D \rightarrow D, i \mid i$

当前栈中元素依次为 $i (i$ 。应当如何规约？

- 如果 i 对应的符号表示一个过程，则应该调用 $L \rightarrow i$ 进行归约；
- 如果 i 对应为一个数组，则应该调用 $D \rightarrow i$ 来归约；

问：老师，看后面有没有 $:=$ 不就可以决定了吗？

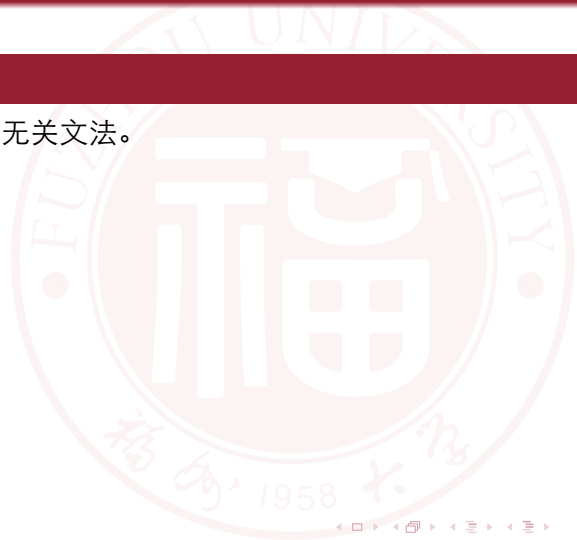


- ① 课前复习
- ② 移入/归约冲突
- ③ LR 分析法
- ④ LR(0)

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。



LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；

LR(k) 文法

定义

- LR(**k**) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - **k** 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？
用哪个产生式归约？

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
- 分析的高效性：最通用的无回溯移入归约分析法；

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
- 分析的高效性：最通用的无回溯移入归约分析法；
- 错误发现早：可以尽可能早的检测到错误；

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
- 分析的高效性：最通用的无回溯移入归约分析法；
- 错误发现早：可以尽可能早的检测到错误；
- 支持的语言是 LL(1) 语言的真超集。

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
- 分析的高效性：最通用的无回溯移入归约分析法；
- 错误发现早：可以尽可能早的检测到错误；
- 支持的语言是 LL(1) 语言的真超集。

LR(k) 文法

定义

- LR(k) 文法是一类特别的上下文无关文法。
 - L 表示 “Left-to-Right”，即从左往右扫描；
 - R 表示 “Rightmost”，即最右推导；
 - k 表示每次推导需要看 k 个输入符号 来决定语法分析动作，即决定什么时候归约？
用哪个产生式归约？

LR 文法的优势：

- 语言的通用性：几乎所有语言都可以使用 LR 文法来定义；
- 分析的高效性：最通用的无回溯移入归约分析法；
- 错误发现早：可以尽可能早的检测到错误；
- 支持的语言是 LL(1) 语言的真超集。

LR 分析器的缺点：实现起来很复杂。

LR 分析器与原始移进-归约框架的区别

对比维度	原始移进-归约框架	LR 分析器
核心机制	简单操作 ²	基于 ACTION/GOTO 表的状态机
冲突处理	无系统方法 需手动解决冲突	通过分析表自动解决冲突 ³
状态管理	无状态概念 仅依赖栈内容	基于 LR 项集的状态管理 (每个状态对应文法的一个分析阶段)
处理能力	仅限极简单文法	可处理复杂左递归、嵌套结构
实现难度	简单但低效	复杂但高效 (需预先生成分析表)
典型工具	无专用工具	Yacc/Bison (基于 LALR(1))

²基于栈和输入符号的 4 种操作

³如 SLR 利用 FOLLOW 集

LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖当前状态决定动作。
- 构造简单，但分析能力有限，容易产生冲突。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。



LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖**当前状态**决定动作。
- 构造简单，但分析能力有限，**容易产生冲突**。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

② SLR(1)（简单 LR）分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，**解决部分冲突**。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。



LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖**当前状态**决定动作。
- 构造简单，但分析能力有限，**容易产生冲突**。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

② SLR(1)（简单 LR）分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，**解决部分冲突**。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。

③ LALR(1)（向前看 LR）分析器

- 基于 LR(1) 项集，但通过合并同心项集减少状态数量。
- 状态数与 SLR(1) 接近，但**处理能力更强**。
- 大多数编程语言的**实际文法**，广泛用于编译器工具（如 Yacc/Bison）。

LR 分析器家族

① LR(0) 分析器

- $k = 0$ ，仅依赖**当前状态**决定动作。
- 构造简单，但分析能力有限，**容易产生冲突**。
- 仅能处理极简单的无冲突文法（如无二义性的规则文法）。

② SLR(1)（简单 LR）分析器

- 在 LR(0) 的基础上，利用 FOLLOW 集缩小归约范围，**解决部分冲突**。
- 向前看 1 个符号，但仅在归约时检查 FOLLOW 集。
- 能处理部分 LR(0) 无法解决的冲突，但对某些文法仍存在局限性。

③ LALR(1)（向前看 LR）分析器

- 基于 LR(1) 项集，但通过合并同心项集减少状态数量。
- 状态数与 SLR(1) 接近，但**处理能力更强**。
- 大多数编程语言的**实际文法**，广泛用于编译器工具（如 Yacc/Bison）。

④ 规范 LR(1) 分析器

- 最强大的 LR 分析器，精确跟踪每个项的向前看符号。
- 状态数远多于 LALR(1)，但能**处理所有 LR(1) 文法**。
- 用于需要处理复杂冲突的文法，但实现成本较高。

LR 家族各项比较

分析器类型	向前看符号	核心方法	处理能力	典型应用
LR(0)	0	仅依赖当前状态	最低	教学示例 极简文法分析
SLR(1)	1 ⁴	利用 FOLLOW 集限制归约	较低	简单编程语言 表达式文法
LALR(1)	1	合并 LR(1) 的同心项集	较高	现代编译器 (如 Yacc/Bison)
规范 LR(1)	1	精确跟踪向前看集合	最高	复杂文法 理论研究

⁴仅归约时使用

LR 家族各自优劣势

分析器类型	状态数	优势	劣势
LR(0)	最少	构造简单， 状态数少	极易产生移进-归约 或归约-归约冲突
SLR(1)	与 LR(0) 相同	冲突处理优于 LR(0)， 实现简单	对 FOLLOW 集外的冲突 仍无法解决
LALR(1)	接近 SLR(1)	状态数少且能力强， 平衡性能与复杂度	可能无法处理某些 规范 LR(1) 文法
规范 LR(1)	最多 ⁵	无冲突，能力最强	状态数庞大 构造和存储成本极高

⁵指数级增长

LR 家族具体分析示例

例

假设文法：

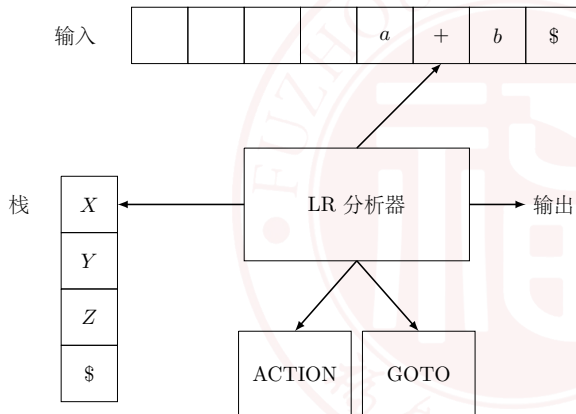
$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

分析器类型	能否无冲突分析	状态数	冲突类型
LR(0)	不能	~10	移进 vs 归约
SLR(1)	能	~10	无冲突
LALR(1)	能	~12	无冲突
规范 LR(1)	能	~30	无冲突

LR 分析器模型图



ACTION 与 GOTO 表

定义

$ACTION[s, a]$: 当状态 s 面临输入符号 a 时, 应采取的动作。

$GOTO[s, X]$: 状态 s 面对文法符号 X 时的下一状态。



- ① 课前复习
- ② 移入/归约冲突
- ③ LR 分析法
- ④ LR(0)

LR(0) 分析器

构成要件

- 项 (item)
- 项集闭包 (closure)
- GOTO 函数

构建流程

- 拓广文法
- 构造项集族
- 构建 LR(0) 分析表

分析算法

- 分析栈结构
- 分析步骤

局限

- 移进-归约冲突
- 归约-归约冲突

项 (item)

LR 分析器通过维护一些**状态**来存储语法分析过程**进度**，用于判定需要移入还是归约。

这些**状态**代表了“项” (item) 的集合。项的集合便称为**项集**。

文法 G 的一个**LR (0) 项 (简称为项)**是 G 的一个产生式再加上一个式体中某处的点。

例

$$A \rightarrow XYZ \Rightarrow \begin{array}{l} A \rightarrow \cdot XYZ \\ A \rightarrow X \cdot YZ \\ A \rightarrow XY \cdot Z \\ A \rightarrow XYZ \cdot \end{array}$$

特别地, $A \rightarrow \varepsilon \Rightarrow A \rightarrow \cdot$ 。

项的意义

项指明了在语法分析过程中已经看到的表达式的部分，其中 $\cdot$ 用于分隔已识别和未识别部分。

例

对产生式 $S \rightarrow aBc$ ，可能的项包括：

$S \rightarrow \cdot aBc$ （未开始识别）

$S \rightarrow a \cdot Bc$ （已识别 a ，待识别 Bc ）

$S \rightarrow aB \cdot c$ （已识别 aB ，待识别 c ）

$S \rightarrow aBc \cdot$ （已完成识别）

项集

项集 I 定义如下:

$$I = \{[A \rightarrow \alpha \cdot X\beta], [X \rightarrow \cdot \gamma], \dots\}$$

规范 LR(0) 项集族

规范 LR(0) 项集族 (canonical LR(0) collection) 是项集的集合，是构建确定性 LR(0) 自动机的基础。



规范 LR(0) 项集族

规范 LR(0) 项集族 (canonical LR(0) collection) 是项集的集合，是构建确定性 LR(0) 自动机的基础。

如何构造？



规范 LR(0) 项集族

规范 LR(0) 项集族 (canonical LR(0) collection) 是**项集的集合**，是构建**确定性 LR(0) 自动机**的基础。

如何构造？

- ① 增广文法 (拓广文法, augmented grammar): 保证 LR 文法分析器有唯一的接受状态;

原始文法:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow id$$

增广文法:

$$E' \rightarrow E$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

⋮

规范 LR(0) 项集族

规范 LR(0) 项集族 (canonical LR(0) collection) 是**项集的集合**，是构建**确定性 LR(0) 自动机**的基础。

构造规范 LR(0) 项集族的主要步骤：

- ① 增广文法（拓广文法，augmented grammar）目的是保证文法有唯一的接受状态；
- ② 通过生成式构造项；
- ③ 闭包（CLOSURE）计算；
- ④ GOTO 计算；
- ⑤ 计算规范 LR(0) 项集族。

项集闭包的计算

Algorithm 1: 项集闭包的计算

Input: 项集 I

Output: 项集闭包 $CLOSURE(I)$

```

1  while true do
2       $I' \leftarrow \emptyset$ ;
3      for  $i \in I$  do
4          if “.” 后紧跟非终结符  $B$  then
5              for  $B$  的所有产生式  $B \rightarrow \gamma$  do
6                  if  $\neg(B \rightarrow \cdot \gamma \in I)$  then
7                       $I' \leftarrow I' \cup \{B \rightarrow \cdot \gamma\}$ ;
8                  end
9              end
10         end
11     end
12     if  $I' = \emptyset$  then
13         break;
14     end
15 end
    
```

项集闭包

例

假设初始项集 $I = \{S' \rightarrow \cdot S\}$ ，且存在 $S \rightarrow abc$ ，则闭包扩展后有

$$\text{CLOSURE}(I) = \{S' \rightarrow \cdot S, S \rightarrow \cdot abc\}$$

⁶无需列出所有产生式，记录一下哪个文法符号即可

项集闭包

例

假设初始项集 $I = \{S' \rightarrow \cdot S\}$ ，且存在 $S \rightarrow abc$ ，则闭包扩展后有

$$\text{CLOSURE}(I) = \{S' \rightarrow \cdot S, S \rightarrow \cdot abc\}$$

如果有一个点在最左端的 B 的产生式被加入项集，则其余 B 的产生式也会被加入项集⁶。

⁶无需列出所有产生式，记录一下哪个文法符号即可

项集闭包

例

假设初始项集 $I = \{S' \rightarrow \cdot S\}$ ，且存在 $S \rightarrow abc$ ，则闭包扩展后有

$$\text{CLOSURE}(I) = \{S' \rightarrow \cdot S, S \rightarrow \cdot abc\}$$

如果有一个点在最左端的 B 的产生式被加入项集，则其余 B 的产生式也会被加入项集⁶。

内核项与非内核项：

- 内核项：初始项 $S' \rightarrow \cdot S$ 和其他点不在左端的所有项；
- 非内核项：除了初始项 $S' \rightarrow \cdot S$ 以外其他点在左端的项。

内核项有进度信息需要存储，非内核项无需记录可以通过闭包运算重新获得。

⁶无需列出所有产生式，记录一下哪个文法符号即可

GOTO 函数

$\text{GOTO}(\mathbf{I}, X) := \text{CLOSURE}(\{A \rightarrow \alpha X \cdot \beta \mid A \rightarrow \alpha \cdot X \beta \in \mathbf{I}\})$

例

输入项集 $\mathbf{I} = \{S \rightarrow a \cdot B, B \rightarrow \cdot c, B \rightarrow \cdot d\}$, 面临符号 B , 则

$$\text{GOTO}(\mathbf{I}, B) = \{S \rightarrow aB \cdot\}$$

GOTO 函数

练习

考虑增广文法

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

给定项集 $I = \{E' \rightarrow E \cdot, E \rightarrow E \cdot + T\}$,
求 $GOTO(I, +)$ 。

GOTO 函数

练习

考虑增广文法

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

给定项集 $I = \{E' \rightarrow E \cdot, E \rightarrow E \cdot + T\}$,
求 $GOTO(I, +)$ 。

解

$$\begin{aligned} GOTO(I, +) &= CLOSURE(\{E \rightarrow E + \cdot T\}) = \{ \\ &E \rightarrow E + \cdot T, \\ &T \rightarrow \cdot T * F, \\ &T \rightarrow \cdot F, \\ &F \rightarrow \cdot (E), \\ &F \rightarrow \cdot id \\ &\} \end{aligned}$$

构造规范 LR(0) 项集族

Algorithm 2: 规范 LR(0) 项集族的计算

```

1  $\mathcal{C} \leftarrow \{\text{CLOSURE}(\{[S' \rightarrow \cdot S]\})\};$ 
2 repeat
3   for  $\mathbf{I} \in \mathcal{C}$  do
4     for  $X \in \mathbb{V}_N \cup \mathbb{V}_T$  do
5       if  $\text{GOTO}(\mathbf{I}, X) \neq \emptyset \wedge \text{GOTO}(\mathbf{I}, X) \notin \mathcal{C}$  then
6          $\mathcal{C} \leftarrow \mathcal{C} \cup \text{GOTO}(\mathbf{I}, X);$ 
7       end
8     end
9   end
10 until 没有新的项集被加入  $\mathcal{C}$ ;
```

构造规范 LR(0) 项集族示例

例

对于增广文法 $S' \rightarrow S; S \rightarrow aB; B \rightarrow c \mid d$, 有

$$I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\}) = \{[S' \rightarrow \cdot S], [S \rightarrow \cdot aB]\}$$

$$I_1 = \text{GOTO}(I_0, S) = \{[S' \rightarrow S \cdot]\}$$

$$I_2 = \text{GOTO}(I_0, a) = \{[S \rightarrow a \cdot B], [B \rightarrow \cdot c], [B \rightarrow \cdot d]\}$$

$$I_3 = \text{GOTO}(I_2, B) = \{[S \rightarrow aB \cdot]\}$$

$$I_4 = \text{GOTO}(I_2, c) = \{[B \rightarrow c \cdot]\}$$

$$I_5 = \text{GOTO}(I_2, d) = \{[B \rightarrow d \cdot]\}$$

LR(0) 自动机的作用

规范归约过程中

- 栈内的符号串和扫描剩下的输入符号串构成了一个规范句型；
- 栈内如果出现句柄，句柄一定在栈的顶部；
- 一旦栈顶出现可归约串 (句柄)，则进行归约。
- 识别了句柄的一部分（即规范句型的活前缀）就相当于识别了当前规范句型的左起部分

因此，我们可以得出一个指导 LR 分析的目标——保证栈中总是**活前缀**。

LR(0) 自动机的作用

规范归约过程中

- 栈内的符号串和扫描剩下的输入符号串构成了一个规范句型；
- 栈内如果出现句柄，句柄一定在栈的顶部；
- 一旦栈顶出现可归约串（句柄），则进行归约。
- 识别了句柄的一部分（即规范句型的活前缀）就相当于识别了当前规范句型的左起部分

因此，我们可以得出一个指导 LR 分析的目标——保证栈中总是**活前缀**。对于文法 G，可构造一个 DFA 来识别 G 的所有活前缀。

LR(0) 自动机

LR(0) 自动机是一个**确定性有穷状态自动机 (DFA)**。



LR(0) 自动机

LR(0) 自动机是一个确定性有穷状态自动机 (DFA) 。

五元组的对应关系：

构成要件	DFA	LR(0) 自动机
字符集	Σ	文法符号 (含终结符和非终结符)
状态	q	一个项集的闭包
有限的状态集合	Q	规范 LR(0) 项集族
状态转移函数	$\delta(q, a)$	GOTO 函数
起始状态	q_0	$CLOSURE(\{[S' \rightarrow \cdot S]\})$
接受状态集合	F	包含项 $[S' \rightarrow S]$ 的项集对应 \$ 输入时的状态

构造文法的 LR(0) 自动机

例

有文法

$$S' \rightarrow S \quad S \rightarrow BB \quad B \rightarrow aB \mid b$$

构造它的 LR(0) 自动机。

构造文法的 LR(0) 自动机-解

解

计算规范 LR(0) 项集族

$$I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\}) = \{[S' \rightarrow \cdot S], [S \rightarrow \cdot BB], [B \rightarrow \cdot aB], [B \rightarrow \cdot b]\}$$

$$I_1 = \text{GOTO}(I_0, S) = \{[S' \rightarrow S \cdot]\}$$

$$I_2 = \text{GOTO}(I_0, B) = \{[S \rightarrow \cdot BB], [B \rightarrow \cdot aB], [B \rightarrow \cdot b]\}$$

$$I_3 = \text{GOTO}(I_0, a) = \text{GOTO}(I_2, a) = \text{GOTO}(I_3, a) = \{[B \rightarrow a \cdot B], [B \rightarrow \cdot aB], [B \rightarrow \cdot b]\}$$

$$I_4 = \text{GOTO}(I_0, b) = \text{GOTO}(I_2, b) = \text{GOTO}(I_3, b) = \{[B \rightarrow b \cdot]\}$$

$$I_5 = \text{GOTO}(I_2, B) = \{[S \rightarrow BB \cdot]\}$$

$$I_6 = \text{GOTO}(I_3, B) = \{[B \rightarrow aB \cdot]\}$$

用 LR(0) 自动机指导移入-归约

状态栈	符号栈	输入	动作
0	\$	id*id\$	转移并移入 5
0 5	\$ id	*id\$	按照 $F \rightarrow id$ 归约
0 3	\$ F	*id\$	按照 $T \rightarrow F$ 归约
0 2	\$ T	*id\$	转移并移入 7
0 2 7	\$ T *	id\$	转移并移入 5
0 2 7 5	\$ T * id	\$	按照 $F \rightarrow id$ 归约
0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
0 1	\$ E	\$	接受

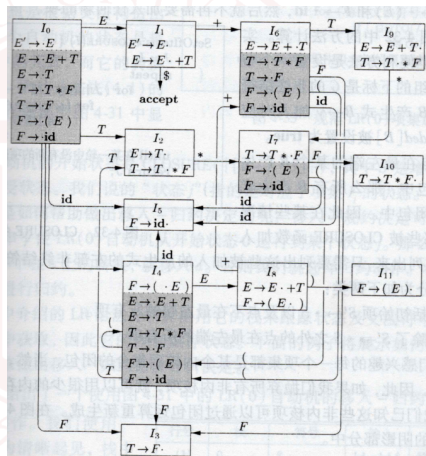


图 4-31 表达式文法(4.1)的 LR(0) 自动机

用 LR(0) 自动机指导移入-归约

状态栈	符号栈	输入	动作
0	\$	id*id\$	转移并移入 5
0 5	\$ id	*id\$	按照 $F \rightarrow id$ 归约
0 3	\$ F	*id\$	按照 $T \rightarrow F$ 归约
0 2	\$ T	*id\$	转移并移入 7
0 2 7	\$ T *	id\$	转移并移入 5
0 2 7 5	\$ T * id	\$	按照 $F \rightarrow id$ 归约
0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
0 1	\$ E	\$	接受

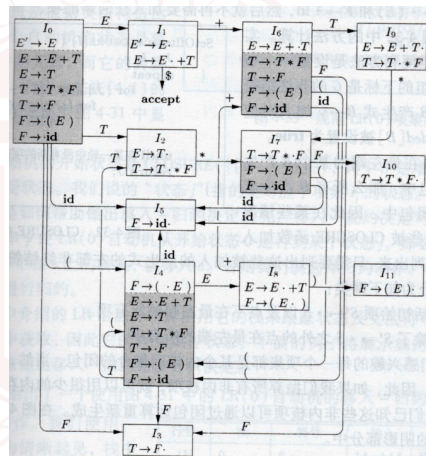


图 4-31 表达式文法(4.1)的 LR(0) 自动机

用 LR(0) 自动机指导移入-归约

状态栈	符号栈	输入	动作
0	\$	id*id\$	转移并移入 5
0 5	\$ id	*id\$	按照 $F \rightarrow id$ 归约
0 3	\$ F	*id\$	按照 $T \rightarrow F$ 归约
0 2	\$ T	*id\$	转移并移入 7
0 2 7	\$ T *	id\$	转移并移入 5
0 2 7 5	\$ T * id	\$	按照 $F \rightarrow id$ 归约
0 2 7 10	\$ T * F	\$	按照 $T \rightarrow T * F$ 归约
0 2	\$ T	\$	按照 $E \rightarrow T$ 归约
0 1	\$ E	\$	接受

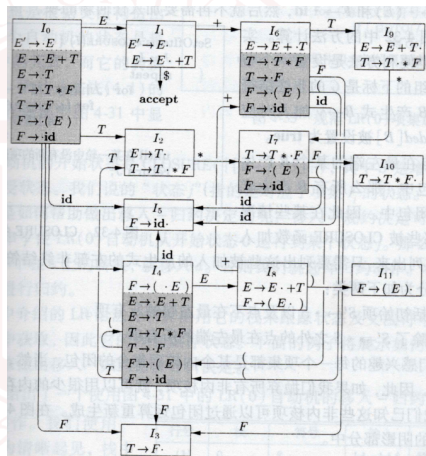


图 4-31 表达式文法(4.1)的 LR(0) 自动机

用 LR(0) 自动机指导移入-归约之直观结论

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；

用 LR(0) 自动机指导移入-归约之直观结论

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；

用 LR(0) 自动机指导移入-归约之直观结论

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；
- 回到第一步，继续判断是否可以归约；

用 LR(0) 自动机指导移入-归约之直观结论

推论

- 当前状态所对应项集闭包中，含有点在最右侧的状态时应当调用归约过程；
- 调用完归约过程后，需要将归约得到的文法符号视作输入继续进行转移；
- 回到第一步，继续判断是否可以归约；
- 状态栈描述了完整的转移流程，可以据此还原符号栈，可以少保存一个符号栈，节约内存；

LR 语法分析表

LR 分析器的语法分析表由两个部分构成：**语法分析动作函数 ACTION**和**转换函数 GOTO**。

语法分析动作函数 ACTION

语法分析动作函数 $ACTION[i, a]$ ，其中 i 为状态和 a 为当前输入 (或终结符 $\$$)，其值有四种形式：

- ① 移入 s_j 。移入状态 j (同时移入符号 a)；
- ② 归约 r_j 。假设第 j 条规则为 $A \rightarrow \beta$ ，则将栈顶的 β 归约为 A ；
- ③ 接受。接受输入串，完成语法分析。
- ④ 报错。在输入中发现一个错误并执行纠正动作。

状态转换函数 GOTO

转换函数 $GOTO[l_i, A]$ ，其中 i 为状态编号和 A 为非终结符。若 $GOTO[l_i, A] = l_j$ ，则在当前状态为 i ，归约得到非终结符 A 时，移入与 A 对应的状态 j 。

LR(0) 分析表

如果基于某文法构建的所有 LR(0) 项集，要么只有移入项目，要么只有~~一个~~归约项目，则可以构建 LR(0) 分析表。同时有：

- 该文法称为 LR(0) 文法；
- 在分析时，不需要看后续输入即可确定动作；
- 分析表中，归约项目无需考虑后继。

LR(0) 分析表的构造

- ① 构造 G' 的规范 LR(0) 项集族 $\mathcal{C} = \{I_0, I_1, \dots, I_n\}$ 。
- ② 令 I_i 对应状态 i , 则 ACTION 表和 GOTO 表构造如下:
 - $(A \rightarrow \alpha \cdot a\beta \in I_i \wedge \text{GOTO}(I_i, a) = I_j) \longrightarrow \text{ACTION}[i, a] = s_j;$
 - $(A \rightarrow \alpha \cdot B\beta \in I_i \wedge \text{GOTO}(I_i, B) = I_j) \longrightarrow \text{GOTO}[i, B] = s_j;$
 - $(A \rightarrow \alpha \cdot \in I_i \wedge A \neq S') \implies \forall a \in \mathbb{V}_T \cup \{\$\} \text{ACTION}[i, a] = r_j$, 其中 j 是产生式 $A \rightarrow \alpha$ 的编号;
 - $(S' \rightarrow S \cdot \in I_i) \implies \text{ACTION}[i, \$] = \text{acc};$
 - 没有定义的所有条目都设置为 “error”。

构造文法的 LR(0) 分析表

例

有文法

$$S' \rightarrow S \quad (1)$$

$$S \rightarrow BB \quad (2)$$

$$B \rightarrow aB \quad (3)$$

$$B \rightarrow b \quad (4)$$

构造它的 LR(0) 分析表。

状态	ACTION			GOTO	
	a	b	\$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

LR(0) 分析法（综合示例）

例

对文法 $E \rightarrow aA \mid bB \quad A \rightarrow cA \mid d \quad B \rightarrow cB \mid d$

试构造 LR(0) 分析表，如果是 LR(0) 文法，分析 bccd 和 bcd。